

# Color Tracking (No-PTZ)

## 1. Experiment Objective

The camera detects an object of a specified color, and the car rotates to track it so that it stays at the center of the frame. It supports Tracking mode (rotation only) and Follow mode (rotation + distance keeping).

## 2. Experiment Principle

### 1. Keyword Explanation

| Keyword            | Description                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Color Tracking     | Detects objects of a specific color based on the HSV color space and tracks their positions                                                    |
| HSV Color Space    | Describes color in three dimensions — Hue (H), Saturation (S), and Value (V) — and is more robust to lighting interference than RGB            |
| Contour Detection  | Finds connected regions in the binarized mask image and takes the largest contour as the target                                                |
| Centroid Deviation | The horizontal pixel offset between the target center and the frame center, used as the input of the PD controller                             |
| PD Control         | Proportional-derivative controller that computes the car's angular velocity output based on the centroid deviation and its rate of change      |
| Deadband           | When the target is within a certain range around the frame center, no steering is triggered, avoiding frequent steering caused by small jitter |
| Distance Following | Controls the car to move forward/backward based on the size change of the target color block radius in the frame to keep a constant distance   |
| QoS                | best_effort (image input + cmd_vel downlink), depth=1                                                                                          |

### 2. Technical Architecture

Dual-timer architecture: image processing and body control are decoupled.

```
/camera/image_raw (BEST_EFFORT)
  → color_tracker_node
    └─ Image processing: image conversion → flip → HSV binarization → morphology
      → largest contour → centroid calculation
        └─ Body control timer (10Hz): pixel error → PD angular velocity + color
          block size distance following → /cmd_vel (BEST_EFFORT)
```

**Mouse HSV sampling:** Drag to select an ROI with the mouse → take the 5th–95th percentile HSV → automatically update the thresholds → lock the target color.

**Tracking mode vs Follow mode:**

- track: `enable_distance_follow:=false`, rotation tracking with PD only
- follow: `enable_distance_follow:=true`, rotation tracking + distance following

### 3. Core Topic Quick Reference

#### Firmware Uplink (Sensor Data)

| Topic                          | Message Type                   | QoS         | Description                  |
|--------------------------------|--------------------------------|-------------|------------------------------|
| <code>/camera/image_raw</code> | <code>sensor_msgs/Image</code> | best_effort | ESP32 camera raw image frame |

#### Firmware Downlink (Control Commands)

| Topic                 | Message Type                     | QoS                        | Field                  | Range            | Direction                                             |
|-----------------------|----------------------------------|----------------------------|------------------------|------------------|-------------------------------------------------------|
| <code>/cmd_vel</code> | <code>geometry_msgs/Twist</code> | best_effort + keep_last(1) | <code>linear.x</code>  | -0.12~0.12 m/s   | <code>+=</code> forward, <code>-=</code> backward     |
|                       |                                  |                            | <code>angular.z</code> | -0.60~0.60 rad/s | <code>+=</code> turn left, <code>-=</code> turn right |

### 3. Experiment Steps

#### 1. Hardware Preparation

| Hardware            | Requirement                    |
|---------------------|--------------------------------|
| PTZ version         | ✗                              |
| No-PTZ version      | ☑                              |
| Required peripheral | ESP32 camera (bracket-mounted) |

#### 2. Start the Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

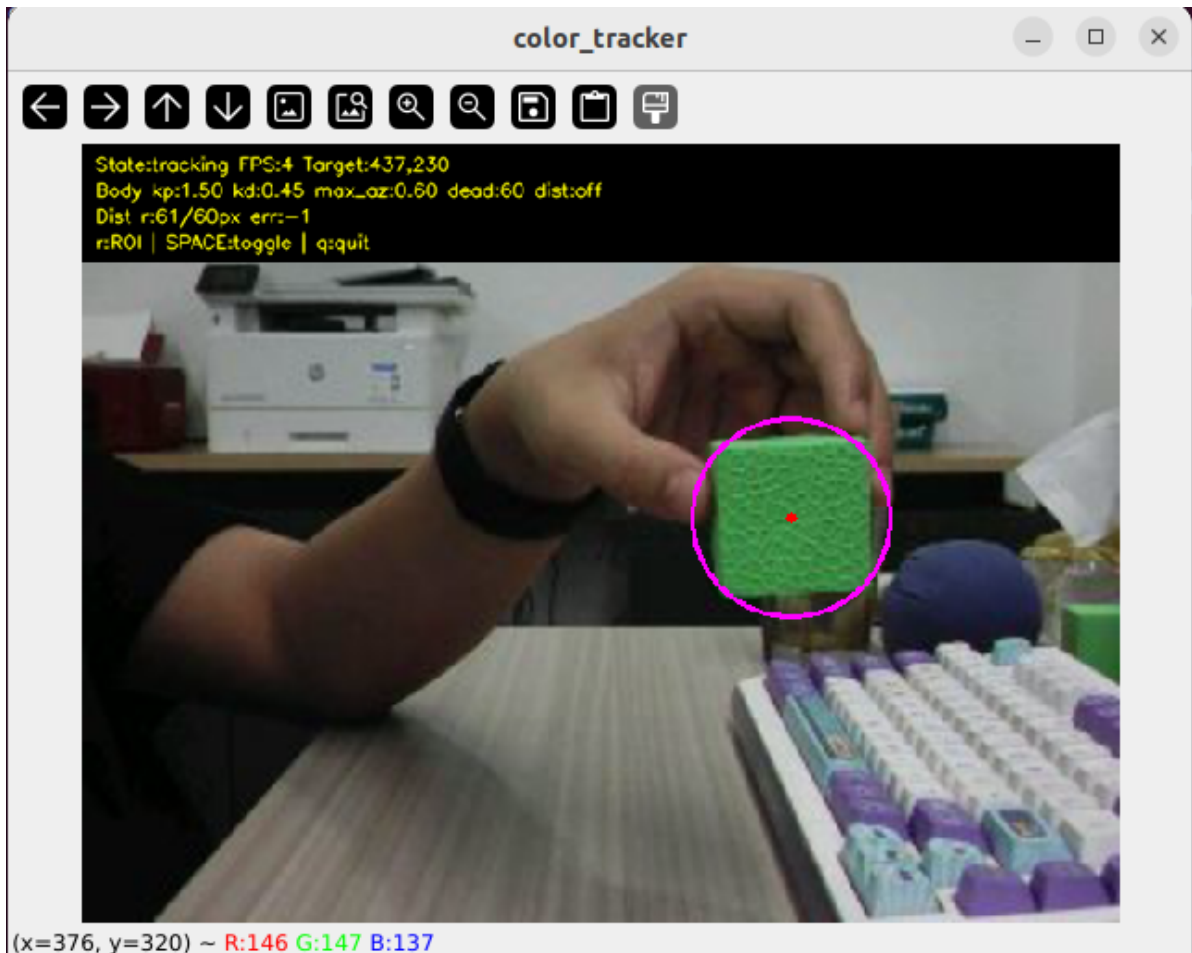
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.785741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

### 3. Start Color Tracking (Terminal 2)

Tracking mode (rotation tracking only):

```
ros2 launch microros_v2_no_ptz_visual_control_pkg color_tracker.launch.py
```



### 4. Operation Instructions

| Key         | Function                                  |
|-------------|-------------------------------------------|
| Space       | Toggle tracking/standby state             |
| r / R       | Re-enable ROI selection (re-sample color) |
| i / I       | Switch to the standby (IDLE) state        |
| q / Q / ESC | Exit the program                          |

| Operation                       | Function                                                                        |
|---------------------------------|---------------------------------------------------------------------------------|
| Drag with the left mouse button | Drag to select the target color area; HSV thresholds are computed automatically |

## 4. Experiment Phenomenon

### In Tracking mode:

- When the target moves left or right in the frame, the car automatically rotates to track it, keeping the target within the center deadband of the frame
- When the target is stationary, the car also stays stationary (zero-velocity command)
- After drag-selecting objects of different colors with the mouse, the HSV thresholds update automatically, enabling color switching

### In Follow mode:

- While rotating to track, the car adjusts its distance to the target based on the size of the target color block
- When the target moves away, the car moves forward to approach it; when the target moves closer, the car backs away

### When the target is lost:

- After consecutive lost frames exceed the threshold, the car stops moving and displays "Target:None"
- When the target reappears in the frame, tracking resumes automatically

## 5. Code Explanation

Source path:

- Launch (track):  
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/color_tracker.launch.py`
- Launch (follow):  
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/color_follower.launch.py`
- Node source:  
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/color_tracker_node.py`

# 1. Core Node Analysis

`color_tracker_node` uses a dual-timer architecture similar to `follow_line_node`. Core workflow:

- **Image caching:** the callback only caches the latest raw Image frame
- **Image processing timer:** image conversion → flip → HSV binarization → morphological opening/closing → largest contour extraction → centroid + radius calculation
- **Body control timer** (10Hz): horizontal pixel error → PD computes angular velocity + color block radius deviation → PD computes linear velocity → `/cmd_vel`

**Constructor — declare parameters, create publishers/subscribers/timers:**

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/color_tracker_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._tgt_lock = threading.Lock()
    self._latest_frame: Optional[np.ndarray] = None

    self._state = STATE_IDLE
    self._shutdown = False

    # Target state: centroid coordinates, color block radius, lost-frame count
    self._has_target = False
    self._lost_frames = 0
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._target_r: Optional[float] = None

    # Mouse HSV sampling state
    self._mouse_selecting = False
    self._roi_start = (0, 0)
    self._roi_end = (0, 0)
    self._roi_pending = False

    self._declare_params()
    self._load_params()

    # Image subscription (BEST_EFFORT, KEEP_LAST, depth=1)
    best_effort = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, best_effort)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, best_effort)

    # Dual timers: image processing + body control
    self._img_timer = self.create_timer(
        1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
        self._process_image)
    self._body_timer = self.create_timer(
```

```

        1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ),
self._body_tick)

self._param_cb = self.add_on_set_parameters_callback(self._on_param_update)

if self._show_win:
    self._ensure_win()

```

### Body control timer callback — PD angular velocity + distance following:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/color_tracker_node.py

def _body_tick(self) -> None:
    """Body control timer (10Hz)."""
    if self._state != STATE_TRACKING or self._shutdown:
        self._pub_zero()
        return

    with self._tgt_lock:
        has = self._has_target
        tx = self._target_x
        tr = self._target_r

    if not has or tx is None:
        self._pub_zero()
        return

    # Horizontal pixel error → PD angular velocity
    cx = self._img_w * 0.5
    err_x = tx - cx
    if abs(err_x) <= self._deadband_x:
        az = 0.0
    else:
        now = time.perf_counter()
        dt = now - self._prev_t if self._prev_t > 0 else 1.0 / self._body_rate
        dt = max(dt, 1e-3)
        d_err = (err_x - self._prev_err) / dt
        self._prev_err = err_x; self._prev_t = now
        az = self._angular_kp * err_x + self._angular_kd * d_err
        az = clamp(az, -self._max_az, self._max_az)
        if self._reverse_body:
            az = -az

    cmd = Twist()
    cmd.angular.z = az

    # Distance following: color block radius deviation → PD linear velocity
    if self._dist_follow and tr is not None:
        err_r = self._target_r_px - tr
        if abs(err_r) > self._dist_tol_r:
            now = time.perf_counter()
            dt = now - self._prev_dt if self._prev_dt > 0 else 1.0 /
self._body_rate
            dt = max(dt, 1e-3)
            d_err_r = (err_r - self._prev_derr) / dt

```

```

self._prev_derr = err_r; self._prev_dt = now
lx = self._dist_kp * err_r + self._dist_kd * d_err_r
lx = clamp(lx, -self._max_lx, self._max_lx)
if self._reverse_linear:
    lx = -lx
cmd.linear.x = lx

self._cmd_pub.publish(cmd)

```

**Key logic:** After drag-selecting an ROI with the mouse, `_roi_pending` triggers `_update_hsv_from_roi()` — it takes the 5th-95th percentile values of the H/S/V channels within the ROI, expands them with the `h_expand` / `s_expand` / `v_expand` tolerances, updates the HSV thresholds, and writes them to the parameter server.

## 2. Key Parameters

Parameter definition file:

- Launch:  
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/color_tracker.launch.py`
- Node:  
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/color_tracker_node.py`

| Parameter                           | Type  | Default                | Description                                                                              |
|-------------------------------------|-------|------------------------|------------------------------------------------------------------------------------------|
| <code>deadband_x_pixel</code>       | float | <code>60.0</code> px   | Horizontal deadband; no steering is triggered when the target is within this pixel range |
| <code>angular_kp</code>             | float | <code>1.5</code>       | Body angle PD proportional coefficient                                                   |
| <code>angular_kd</code>             | float | <code>0.45</code>      | Body angle PD derivative coefficient                                                     |
| <code>max_angular_z</code>          | float | <code>0.6</code> rad/s | Maximum body steering speed                                                              |
| <code>lost_frame_threshold</code>   | int   | <code>5</code>         | Consecutive lost-frame tolerance threshold                                               |
| <code>reverse_body_direction</code> | bool  | <code>true</code>      | Whether to reverse the body steering direction                                           |
| <code>enable_distance_follow</code> | bool  | <code>false</code>     | Whether to enable distance following (track=false, follow=true)                          |
| <code>target_radius</code>          | float | <code>60.0</code> px   | Target color block radius for distance following                                         |
| <code>distance_tolerance_r</code>   | float | <code>12.0</code> px   | Distance-following deadband                                                              |
| <code>distance_kp</code>            | float | <code>0.008</code>     | Distance PD proportional coefficient                                                     |
| <code>distance_kd</code>            | float | <code>0.003</code>     | Distance PD derivative coefficient                                                       |
| <code>max_linear_x</code>           | float | <code>0.12</code> m/s  | Maximum forward/backward speed                                                           |
| <code>body_control_rate</code>      | float | <code>10.0</code> Hz   | Body control rate                                                                        |
| <code>Hmin/Smin/Vmin</code>         | int   | <code>0/85/126</code>  | HSV lower thresholds (auto-updated after mouse sampling)                                 |
| <code>Hmax/Smax/Vmax</code>         | int   | <code>9/253/253</code> | HSV upper thresholds (auto-updated after mouse sampling)                                 |
| <code>morph_kernel_size</code>      | int   | <code>5</code>         | Morphological operation kernel size                                                      |

### 3. Node Description

| Node                            | Package                                            | Description                                                            |
|---------------------------------|----------------------------------------------------|------------------------------------------------------------------------|
| <code>color_tracker_node</code> | <code>microros_v2_no_ptz_visual_control_pkg</code> | HSV color tracking with dual timers + PD body control + mouse sampling |

### 6. Common Problems



| Problem                        | Cause                                                   | Solution                                                                              |
|--------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| Tracking direction is reversed | <code>reverse_body_direction</code> is misconfigured    | Toggle <code>reverse_body_direction</code> to the opposite value                      |
| Target is not detected         | HSV thresholds do not match or lighting changes         | Drag-select the target object again with the mouse to sample HSV                      |
| The car jitters frequently     | The deadband <code>deadband_x_pixel</code> is too small | Increase the deadband value appropriately (e.g., change 60 to 80)                     |
| Poor distance-following effect | Distance PD parameters are unreasonable                 | Tune <code>distance_kp</code> / <code>distance_kd</code> / <code>target_radius</code> |